

- JS4-数组
 - 1 数组创建和初始化
 - 1.1 使用字面量创建和初始化数组
 - 1.2 使用构造函数创建和初始化数组
 - 2 数组属性
 - length
 - 3 常用方法
 - 3.1 可变方法 (Mutation)
 - 修改元素方法
 - splice
 - fill
 - 改变顺序方法
 - reverse
 - sort
 - 3.2 不可变方法 (Immutable)
 - 转换方法
 - join
 - toString
 - concat
 - slice
 - flat
 - 总结
 - 循环迭代方法
 - forEach
 - map
 - filter
 - reduce
 - every
 - some
 - 总结
 - 查找检索方法
 - indexOf
 - includes
 - find
 - at

JS4-数组

在JavaScript中，数组 **Array** 是一种非常重要的数据结构，它允许我们使用单一的变量名来存储一系列的值。数组可以包含多种类型的元素，包括字符串、数字、对象甚至其他数组。数组提供了一系列的方法和属性，使得对数组的操作变得简单高效。

1 数组创建和初始化

1.1 使用字面量创建和初始化数组

使用字面量是创建和初始化数组的最常见和简单的方式。你可以直接在代码中使用方括号 `[]` 来定义一个数组，并在方括号内使用逗号 `,` 分隔每个元素。

```
// 创建一个空数组
const arr = [];
// 创建并初始化一个包含多个元素的数组
const fruits = ['apple', 'banana', 'orange'];
// 创建并初始化一个多维数组
const matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
```

1.2 使用构造函数创建和初始化数组

除了使用字面量，你还可以使用数组的构造函数 **Array()** 来创建和初始化数组。构造函数接受可选的参数来指定数组的长度或初始化数组的元素。

```
// 使用构造函数创建一个空数组
var arr = new Array();
// 使用构造函数创建并初始化一个数组
var numbers = new Array(1, 2, 3, 4, 5);
// 使用构造函数创建指定长度的数组
var lengthArray = new Array(5); // 创建一个长度为 5 的空数组
// 创建一个长度为1的数组，并将数组的元素值填充为5
var lengthArray = new Array(1).fill(5);
console.log(lengthArray); // [5]
// 创建并初始化一个多维数组
var matrix = new Array([1, 2, 3], [4, 5, 6], [7, 8, 9]);
```

2 数组属性

length

设置或返回数组元素的个数。

```
// 设置数组元素的个数
const arr = new Array();
arr.length = 5;
console.log(arr);           // [ <5 empty items> ]

// 返回数组元素的个数
const fruits = ['apple', 'banana', 'orange'];
console.log(fruits.length); // 3
```

3 常用方法

3.1 可变方法（Mutation）

修改元素方法

方法	功能
push	向数组末尾添加一个或多个元素，并返回新数组的长度
pop	删除数组的最后一个元素，并返回被删除的元素
unshift	向数组开头添加一个或多个元素，并返回新数组的长度
shift	删除数组的第一个元素，并返回被删除的元素

splice

- **功能**：向数组中添加/删除/替换元素
- **参数**：`splice(start[, deleteCount[, item1[, item2[, ...]]]])` 起始索引，删除数量，要插入的元素
- **返回值**：被删除的元素组成的数组

fill

- **功能**：指定范围内的所有元素替换为固定值
- **参数**：`fill(value[, start[, end]])` 填充值，起始索引，结束索引
- **返回值**：修改后的原数组

改变顺序方法

reverse

- **功能**：反转数组中元素的顺序
- **参数**：无
- **返回值**：反转后的原数组

sort

- **功能**：对数组元素进行排序，默认按字符串Unicode编码排序
- **参数**：可选的比较函数 $(a, b) \Rightarrow a - b$ （升序）或 $b - a$ （降序）
- **返回值**：排序后的原数组

3.2 不可变方法（Immutable）

转换方法

join

`join()` 方法用于把数组中的所有元素转换一个字符串。元素是通过指定的分隔符进行分隔的。

toString

`toString()` 方法可把数组转换为字符串，并返回结果。**注意**：数组中的元素之间用逗号分隔。

concat

`concat()` 方法用于连接两个或多个数组。该方法不会改变现有的数组，而是返回一个新的数组。

```
const arr1 = ['a', 'b', 'c'];
const arr2 = ['e', 'f'];
const arr3 = ['g'];
```

```
const arr = arr1.concat(arr2, arr3);
console.log(arr);    // [ 'a', 'b', 'c', 'e', 'f', 'g' ]
```

slice

slice() 方法可从已有的数组中返回选定的元素。slice() 方法可提取字符串的某个部分，并以新的字符串返回被提取的部分。**注意：**slice() 方法不会改变原始数组。返回的是数组的**浅拷贝**。

```
// array.slice(start, end)
// 返回一个新的数组，包含从 start（包括该元素） 到 end（不包括该元素）的 arrayObject 中的元素。
const arr = [ 'a', 'b', 'c', 'e', 'f', 'g' ];
console.log(arr.slice(1));    // [ 'b', 'c', 'e', 'f', 'g' ]
console.log(arr.slice(1, 4)); // [ 'b', 'c', 'e' ]
// 使用负值从数组中读取元素
// 截取最后三个元素
console.log(arr.slice(-3));    // [ 'e', 'f', 'g' ]
// 截取倒数第三个（包含）到倒数第一个（不包含）的两个元素
console.log(arr.slice(-3,-1)); // [ 'e', 'f' ]

// 返回的是数组的浅拷贝
const arr1 = [ 'a', 'b', ['c', 'e'], [['f']], 'g' ];
const arr2 = arr1.slice(0);
console.log(arr2); // [ 'a', 'b', [ 'c', 'e' ], [ [ 'f' ] ], 'g' ]
arr1[2][0] = 'g';
console.log(arr2); // [ 'a', 'b', [ 'g', 'e' ], [ [ 'f' ] ], 'g' ]
```

flat

选择性展开嵌套数组。flat([depth]) 可选的展开深度（默认1）。返回值为展开嵌套数组的新数组。

```
const arr = [1, [2, 3, 4], [[5, [6]]]];
const arr1 = arr.flat(1);
console.log(arr1); // [ 1, 2, 3, 4, [ 5, [ 6 ] ] ]
const arr2 = arr.flat(Infinity);
console.log(arr2); // [ 1, 2, 3, 4, 5, 6 ]
```

总结

join() 方法和toString() 方法

```
const arr = [1,2,3,4,5];
console.log(arr.toString());
console.log(arr.join(','));
console.log(arr.join());
// 这三种写法输出值相同
```

循环迭代方法

forEach

forEach() 方法用于调用数组的每个元素，并将元素传递给回调函数。**注意:** forEach() 对于空数组是不会执行回调函数的。

```
// array.forEach(callbackFn(currentValue, index, arr), thisValue)
const numbers = [4, 9, 16, 25];
numbers.forEach(
  (val) => {
    if (val === 16) return; // 跳过元素16
    console.log(val);
  }
)
// 4 9 25
```

map

map() 创建一个新数组，数组中的每个元素是原数组元素调用函数处理后的值。map() 方法按照原始数组元素顺序依次处理元素。**注意:** map() 不会对空数组进行检测。map() 不会改变原始数组。

```
// array.map(function(currentValue,index,arr), thisValue)
const numbers = [4, 9, 16, 25];
const num_sqrt = numbers.map(Math.sqrt);
console.log(num_sqrt); // [ 2, 3, 4, 5 ]

const arr = numbers.map(
  (val) => {
    return val * 2;
  }
)
console.log(arr); // [ 8, 18, 32, 50 ]
```

filter

filter() 方法创建一个新的数组，新数组中的元素是通过检查指定数组中符合条件的所有元素。**注意：**filter() 不会对空数组进行检测。filter() 不会改变原始数组。

```
// array.filter(function(currentValue,index,arr), thisValue)
const ages = [0, 14, 19, 16, 25, 39, 75];
let ages_check = ages.filter(
  (age) => {
    if (age >= 18) return age;
  }
)
console.log(ages_check);    // [ 19, 25, 39, 75 ]

function checkAdult(age) {
  return age >= 18;
}
ages_check = ages.filter(checkAdult);
console.log(ages_check);    // [ 19, 25, 39, 75 ]
```

reduce

reduce() 方法接收一个函数作为累加器，数组中的每个值（从左到右）开始缩减，最终计算为一个值。

```
// array.reduce(function(total, currentValue, currentIndex, arr), initialValue)
const sum = [1,2,3,4,5,6].reduce((a,b)=>a+b,0);
console.log(sum);    // 21
```

every

在数组中所有元素都符合条件，就返回true。

some

在数组中只要有一个元素符合条件，就返回true。

总结

函数	功能	返回值
forEach	遍历	无/undefined
map	转换	新数组（与旧数组等长）

函数	功能	返回值
filter	筛选	新数组（长度小于等于旧数组）
reduce	归约	累积结果
every	全部满足	布尔值
some	存在满足	布尔值

查找检索方法

indexOf

查询并返回数据的索引。

```
console.log([1,3,4].indexOf(4)); // 符合条件的第一个索引 2
console.log([1,4,5,3].indexOf(4,1)); // 索引1开始的元素中索引1对应的是4，返回1
console.log([1,4,5,3].indexOf(4,2)); // 索引2开始的元素中没有4，返回-1
```

includes

检查数组是否包含指定的元素。

find

返回数组中满足条件的第一个元素。

at

通过索引（支持负数）获取数组元素。